

becomes associated with the terminal session that launched it. Linux will terminate once the process you log out of the terminal session. However, if you need to keep a process running even after logging out of the terminal session, you can use the 'nohup' command ('no hangup'). This command will not be associated with the terminal session and will continue running even after you log out. To use the 'nohup' command, type 'nohup' followed by the command you want to execute. For example:

```
$nohup ./longprocess &
```

Because of the 'nohup' command, any output from the process, including error messages, will be redirected to a file called 'nohup.out' in the current directory. You can run commands that take a long time to complete or need to continue running in the background, and ensure these are not terminated when you log out by using the 'nohup' command.

```
[sn@localhost ~]$ nohup ./job1 &
[1] 3965317
[sn@localhost ~]$ nohup: ignoring input and appending output
to 'nohup.out'
[sn@localhost ~]$
```

Job ID versus process ID

In Linux, a process is a program that is currently executing, whereas a job refers to a process or a group of processes managed by a shell. A job is a way of managing processes that run for a long time within a process. Jobs allow you to use job control commands to perform various operations on them. When a job is initiated in a terminal session, it receives a unique job ID (JID). The JID is used to identify the job and manage its processes. The process ID (PID) is not the same as the JID. The JID is used to manage the job while the PID is used to manage individual processes within the job. A job can comprise multiple processes running in sequence, or in parallel, so using the job ID is easier than tracking the individual processes.

You can control the job as a unit, and suspend, resume, or terminate it as a group using job control commands. In contrast, process commands are used to manage the processes individually.

Moving a process from the foreground to the background and vice-versa

By default, a process launched from the command line runs in the foreground. It remains attached to the terminal session, displaying its output in the terminal window. However, if you want to move a process to the background, allowing you to continue using the terminal session, you can use the '&'

symbol at the end of the command. For example, to initiate a process called 'longprocess' in the background, enter the following command:

```
$/longprocess &
```

This command starts the process in the background and returns control of the terminal session to you, as shown below:

```
[sn@localhost ~]$ ./longprocess &
[3] 20721
[sn@localhost ~]$
```

If you have already started a process in the foreground and wish to move it to the background, you can use the 'Ctrl+Z' key combination to suspend the process.

You can then use the 'bg' command ('background') to move it to the background. This command resumes a suspended process in the background. The shell assigns a new job to the process and adds it to the list of background jobs.

```
$bg
```

```
[sn@localhost ~]$ ./longprocess2
^Z
[1]+  Stopped                  ./longprocess2
[sn@localhost ~]$ bg
[1]+ ./longprocess2 &
[sn@localhost ~]$
```

The shell displays the last character '&' (an ampersand) to show that the job is running in the background. By default, the background command moves the most recently suspended process to the foreground. You can have multiple processes suspended at any point in time. To differentiate the multiple processes, append the job number prefixed with a per cent sign (%) on the command line.

To move the first suspended process to the background, use the following command:

```
$bg %1
```

To bring a process back to the foreground, the 'fg' command is employed. This 'fg' command stands for 'foreground' and is used to bring a background process to the foreground. By default, the process that was most recently suspended or moved to the background moves to the foreground. You can also specify which job you want to make as a foreground process by specifying its job number.

```
$fg %1
```